Factorización de un módulo RSA de 256 bits

C3TF - Cyber Competition 2026 • Sede: atlanTTic (Vigo)

Categoría: Crypto

1 El reto

Se entrega el script que cifra la *flag* con RSA y un fichero con los valores públicos n, e y el texto cifrado. El cifrado es RSA estándar ($\texttt{enc} = m^e \bmod n$), sin ningún truco adicional. La debilidad reside en cómo se generan los primos, visible en el Código 1.

Código 1: Generación de la clave (fragmento del reto).

```
p = getPrime(128) # primo de 128 bits
q = getPrime(128) # primo de 128 bits
e = 65537
n = p * q # modulo de 256 bits
enc = pow(int.from_bytes(flag, "big"), e, n)
```

Los valores públicos entregados son:

Código 2: Fichero con los datos del reto.

```
n: 807571030989232378531514558323308994425722294771293106076733498730144172284 21
e: 65537
enc: 536805723304529196950360811916810532883935444767330285758587081923560995748 90
```

2 La vulnerabilidad

La seguridad de RSA se fundamenta en que factorizar $n = p \cdot q$ sea inviable, algo que solo se cumple con módulos grandes (2048 bits o más). Aquí p y q son de 128 bits, de modo que n es de **256 bits** (77 dígitos decimales). Un módulo de ese tamaño es perfectamente factorizable con las herramientas correctas en cuestión de minutos.

Factorizado n, se recuperan p y q, y con ellos el descifrado es el procedimiento habitual de RSA: $\varphi = (p-1)(q-1)$, $d = e^{-1} \bmod \varphi$ y $m = \texttt{enc}^{\,d} \bmod n$. La señal para reconocer el caso es el tamaño de n: si ronda los 77 dígitos o menos, es atacable mediante factorización directa.

3 Explotación

El primer intento fue consultar **FactorDB**, una base de datos de factorizaciones ya calculadas que, cuando el valor está registrado, lo devuelve al instante. Sin embargo, este n no figuraba en ella (aparecía como compuesto, sin factores conocidos), de modo que no sirvió para resolver el reto y hubo que factorizarlo manualmente.

La factorización se consiguió con **PARI/GP** en local, que incluye una criba cuadrática en C. El primer intento, con `factor(n)` directamente, falló: la pila de memoria por defecto (unos 8 MB) es insuficiente para un cálculo de este tamaño y PARI aborta con un error de desbordamiento de pila (*the PARI stack overflows*). Fue necesario ampliar ese límite con `default(parisizemax, "2G")`, que permite a la pila crecer según lo requiera la operación. Con la pila ampliada, `factor(n)` devolvió los dos primos en un par de minutos:

```
prodigy@vesper:~$ gp
Reading GPRC: /etc/gprc
GPRC Done.

                            GP/PARI CALCULATOR Version 2.17.4 (released)
                    amd64 running linux (x86-64/GMP-6.3.0 kernel) 64-bit version
                      compiled: Jul  8 2026, gcc version 16.1.1 20260515 (GCC)
                              threading engine: pthread, nbthreads = 12
                           (readline v8.3 enabled, extended help enabled)

                                Copyright (C) 2000-2024 The PARI Group

PARI/GP is free software, covered by the GNU General Public License, and comes WITHOUT ANY WARRANTY
WHATSOEVER.

Type ? for help, \q to quit.
Type ?18 for how to get moral (and possibly technical) support.

parisize = 8000000, primelimit = 1048576, factorlimit = 1048576
? factor(807571030989232378531514558323308994425722294771293106076733498730144172284 21)
  ***   at top-level: factor(807571030989232378531514558323308994425
  ***                 ^-----------------------------------------------
  *** factor: the PARI stack overflows !
  current stack size: 8000000 (7.629 Mbytes)
  [hint] set 'parisizemax' to a nonzero value in your GPRC

  ***   Break loop: type 'break' to go back to GP prompt
break> break

?
?    default(parisizemax, "2G")
  ***   Warning: new maximum stack size = 2000003072 (1907.352 Mbytes).
?    factor(807571030989232378531514558323308994425722294771293106076733498730144172284 21)
  *** factor: Warning: increasing stack size to 16000000.
  *** factor: Warning: increasing stack size to 32000000.
  *** factor: Warning: increasing stack size to 64000000.
%1 =
[266389670685446299301982514011839137549 1]

[303154033304397379996687987857088285529 1]

? 
```

Figura 1: Sesión de PARI/GP: el primer `factor(n)` falla por desbordamiento de pila; tras `default(parisizemax, "2G")` la pila crece automáticamente y se obtienen los dos primos.

La factorización obtenida es:

$$p = 266389670685446299301982514011839137549$$
$$q = 303154033304397379996687987857088285529$$

Con p y q, el descifrado completo se reduce al Código 3.

Código 3: Descifrado una vez factorizado `n` (`solve.py`).

```python
from Crypto.Util.number import long_to_bytes

p = 266389670685446299301982514011839137549
q = 303154033304397379996687987857088285529
e = 65537
c = 536805723304529196950360811916810532883935444767330285758587081923560995748 90

n = p * q
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi) # clave privada
m = pow(c, d, n) # descifra el mensaje
print(long_to_bytes(m))
```

Al ejecutarlo se obtiene la *flag*:

Código 4: Salida.

```
b'C3TF{128_b1ts_d4n_p4r4_muy_p0c0}'
```

El propio texto de la *flag* lo confirma: 128 bits por primo dan para muy poco.

4 Mitigación

El fallo no está en el algoritmo RSA sino en el tamaño de la clave. La corrección es usar primos lo suficientemente grandes para que n no sea factorizable: el estándar actual es n de al menos 2048 bits, es decir, primos de 1024 bits.

Código 5: Tamaño de clave adecuado.

```
p = getPrime(1024)
q = getPrime(1024)
```